Chapitre 4:

Méthodes d'analyse statistique

Régression Linéaire

Analyse en Composantes Principales

4.1. Régression Linéaire Simple

Qu'est-ce que la régression ?

- Il s'agit de modèles statistiques permettant d'explorer la relation entre une variable réponse et des variables explicatives.
- Connaissant les valeurs des variables explicatives, on peut prédire les valeurs de la variable réponse.

- Variable réponse (ou variable dépendante) :
  - La variable que l'on souhaite prédire.
- Variables explicatives (ou variables indépendantes) :
  - Les variables qui expliquent comment la variable réponse évolue.

Régression linéaire et régression logistique

- **Régression linéaire**
  - La variable réponse est **numérique.**
  - Elle permet de prédire une valeur numérique (ex. : prix d'une maison, salaire).
- **Régression logistique**
  - La variable réponse est **binaire** : elle prend deux valeurs possibles (ex. : Oui/Non, Succès/Echec).
  - Elle permet d'estimer la **probabilité** qu'une observation appartienne à une classe (ex. : probabilité qu'un client quitte un service).
  - Un seuil (souvent 0,5) peut être utilisé pour attribuer une classe à partir de la probabilité estimée.

Régression simple et régression multiple

- **Régression simple**
  - Une seule variable explicative est utilisée pour prédire la variable réponse.
  - **Exemple** : prédire le prix d'une maison à partir de sa superficie.
- **Régression multiple**
  - Plusieurs variables explicatives sont utilisées pour prédire la variable réponse.
  - **Exemple** : prédire le prix d'une maison à partir de sa superficie, de son emplacement et de son nombre de chambres.

Régression linéaire

- Un modèle de régression linéaire simple est de la forme:

$$Y = \beta_0 + \beta_1 . X + \epsilon$$

- Où:
  - Y est la variable dépendante (variable réponse)
  - β_0 et β_1 sont les coefficients (ordonnée a l'origine et pente)
  - X est la variable indépendante (variable explicative)
  - ϵ est une erreur aléatoire.

Quelle est la variable réponse ?

- La régression permet de prédire les valeurs d'une variable réponse à partir des valeurs connues des variables explicatives. Le choix de la variable réponse dépend de la question posée, mais dans de nombreux jeux de données, une variable s'impose naturellement.
- Nous explorerons un jeu de données immobiliers taïwanais comportant quatre variables.
  - *dist_to_mrt_station_m* : Distance à la station de métro la plus proche (en mètres).
  - *n_convenience* : Nombre de supérettes de proximité accessibles à pied.
  - *house_age_years* : Âge de la maison (en années, regroupé en trois catégories).
  - *price_twd_msq* : Prix du mètre carré (en dollars taïwanais).

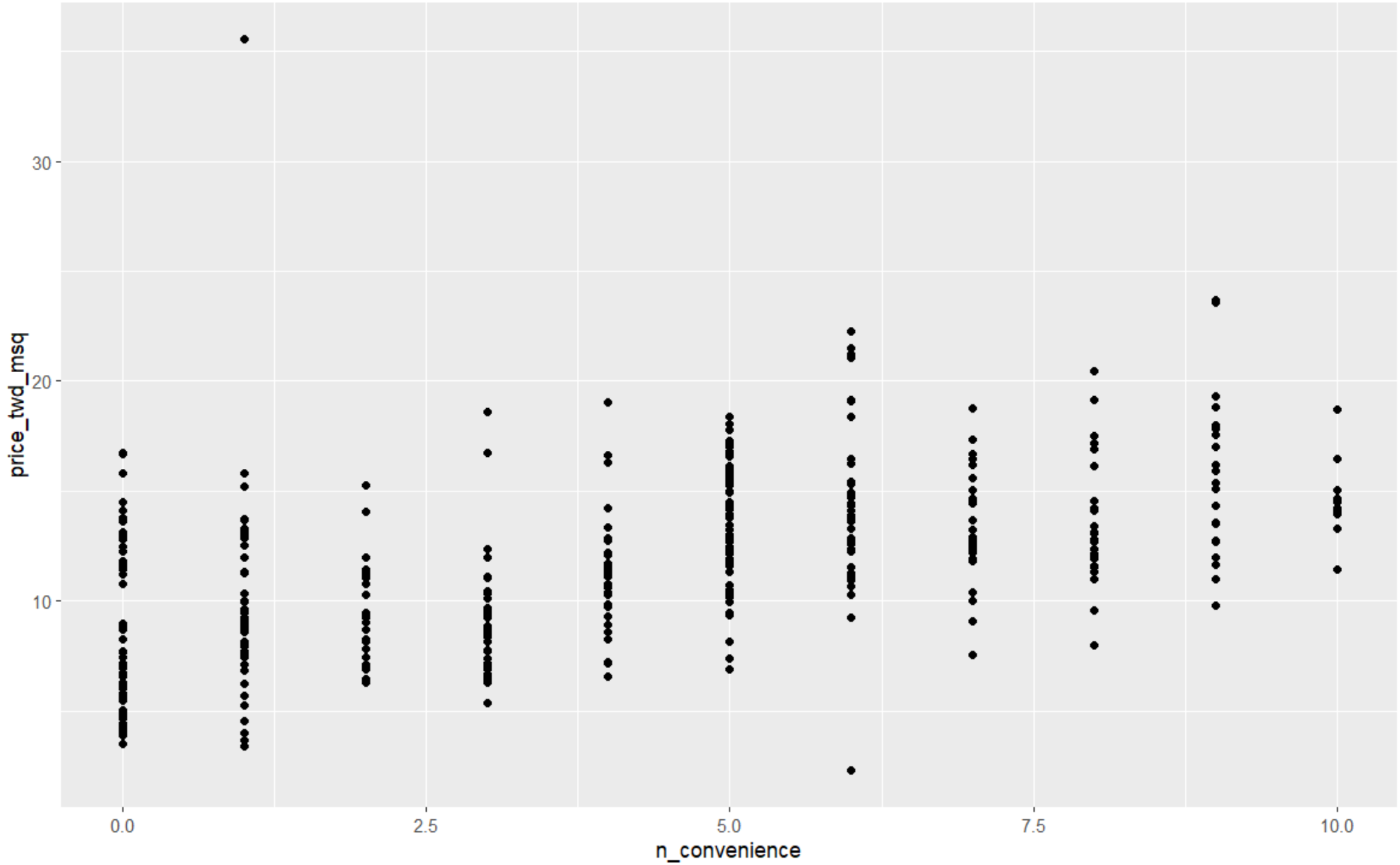
Visualisation de deux variables

- Avant d'appliquer des modèles statistiques, il est généralement conseillé de visualiser vos données. Ici, nous examinerons la relation entre le prix des maisons par unité de surface et le nombre de supérettes à proximité
- En utilisant *taiwan_real_estate*, tracez un nuage de points de price_twd_msq (axe y) en fonction de n_convenience (axe x).

```
library(ggplot2)

ggplot(taiwan_real_estate, aes(x = n_convenience, y
= price_twd_msq)) +
      geom_point()
```

Visualisation de deux variables



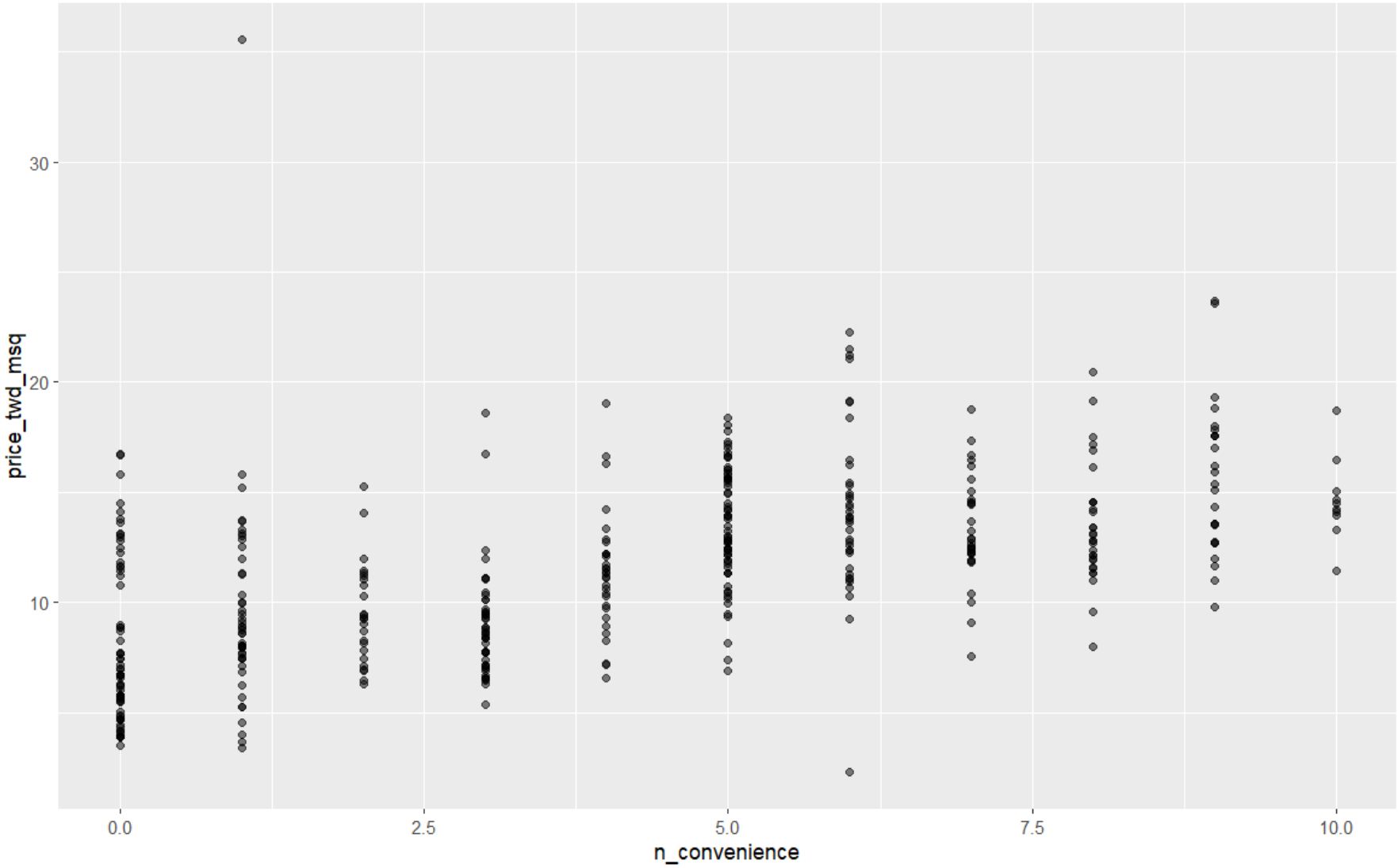
Visualisation de deux variables

- L'une des difficultés de ces données réside dans le fait que le nombre de supérettes est constitué de valeurs entières, ce qui entraîne un chevauchement des points. Pour y remédier, nous allons rendre les points transparents.

```
# Rendre les points transparents à 50%

ggplot(taiwan_real_estate, aes(x = n_convenience, y
= price_twd_msq)) +
      geom_point(alpha = 0.5)
```

Visualisation de deux variables



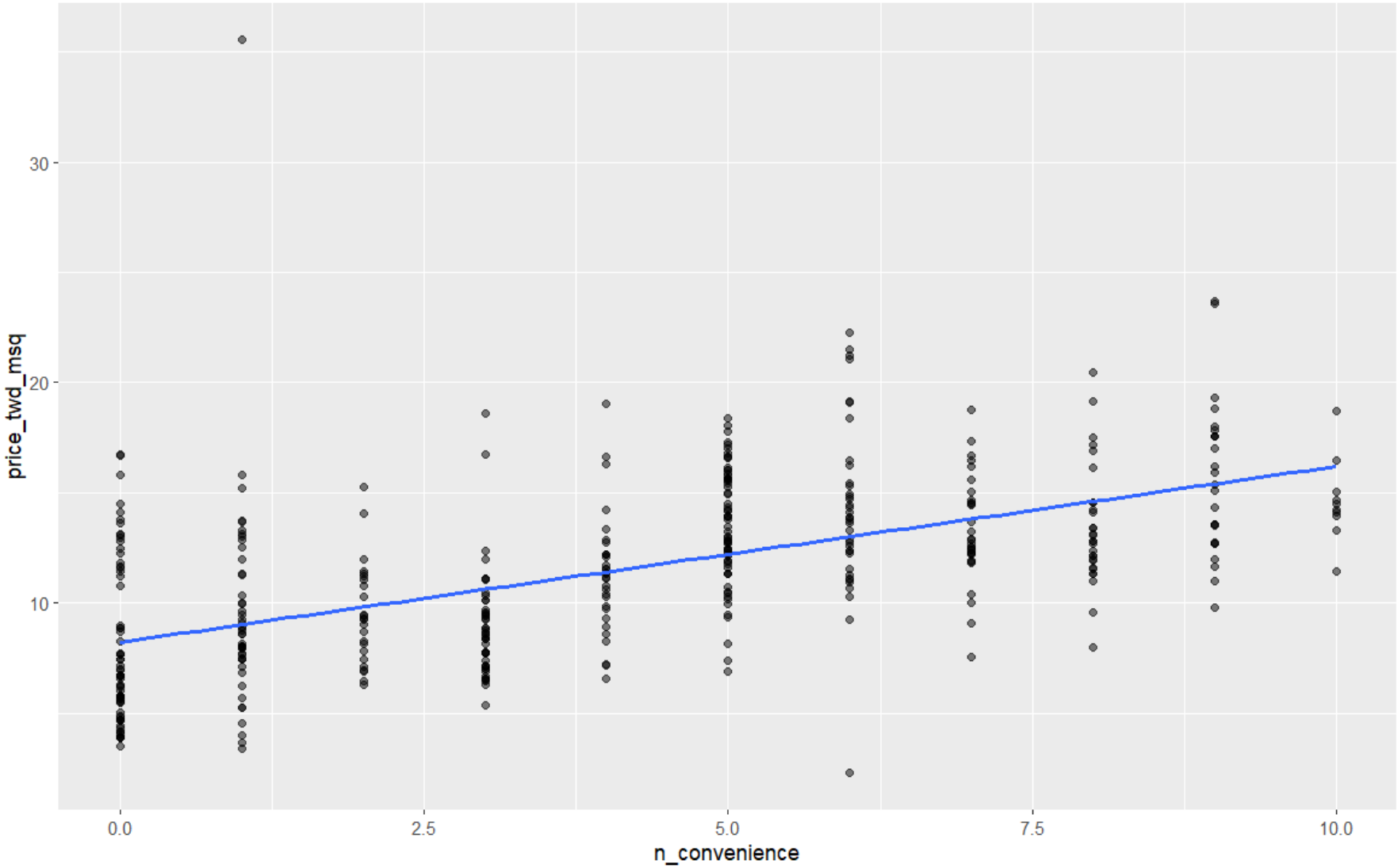
Visualisation de deux variables

- On peut mettre à jour le graphique en ajoutant une courbe de tendance, calculée à l'aide d'une régression linéaire. On peut omettre le ruban de confiance.

```
# Ajouter une courbe de tendance linéaire sans
ruban de confiance

ggplot(taiwan_real_estate, aes(x = n_convenience, y
= price_twd_msq)) +
      geom_point(alpha = 0.5) +
      geom_smooth(
            method = "lm",
            se = FALSE
            )
```

Visualisation de deux variables



Ajustement d'une régression linéaire

- La ligne droite est définie par deux éléments :
  - l'ordonnée à l'origine (*intercept*): la valeur de y lorsque x s'annule.
  - la pente (*slope*): la valeur d' augmentation de y lorsque x augmente de 1.
- Equation :

$$y = intercept + slope \times x$$

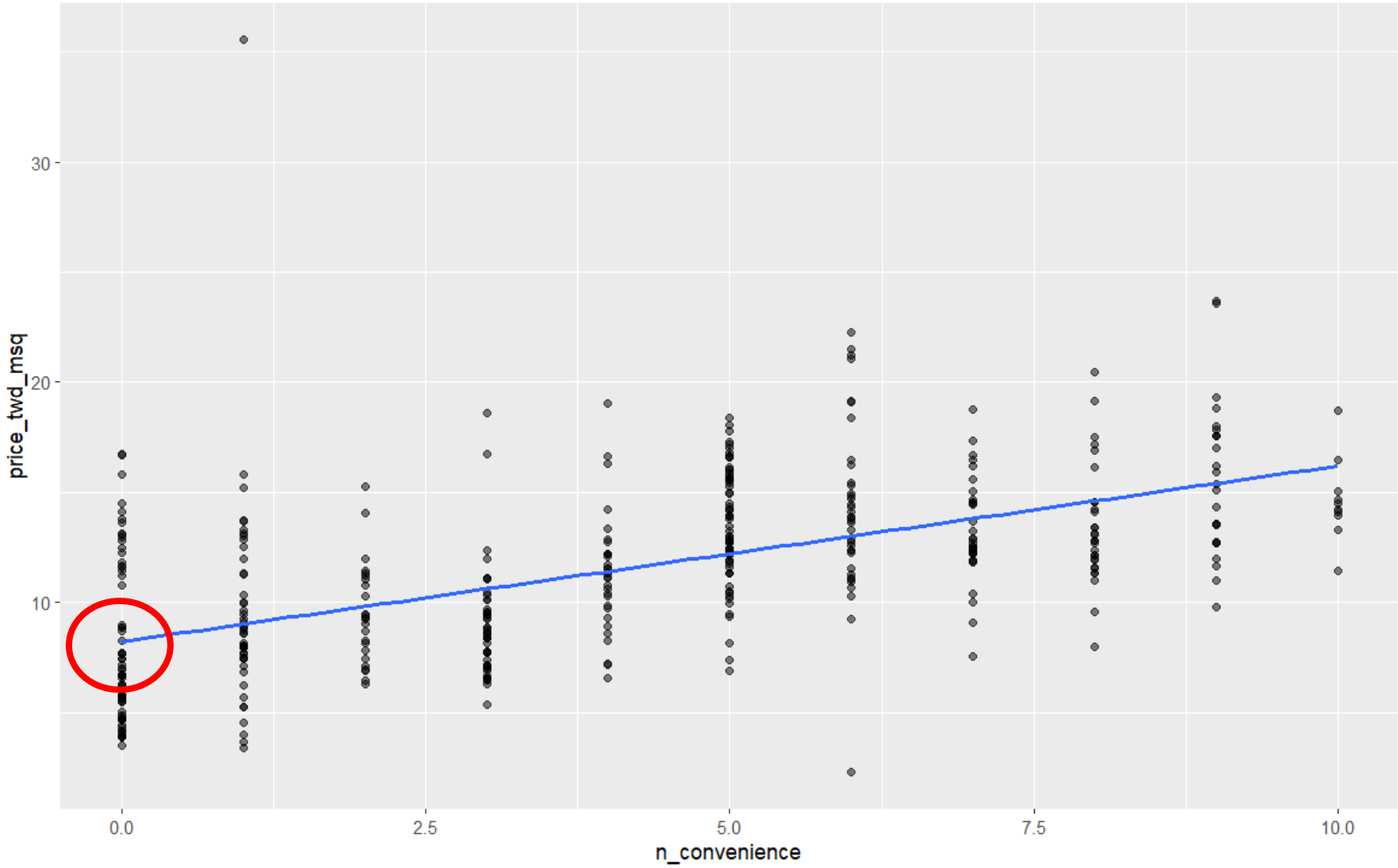
Estimation de l'ordonnée à l'origine (*intercept*)

- Les modèles de régression linéaire ajustent toujours une droite aux données. Les droites sont définies par deux propriétés : leur ordonnée à l'origine et leur pente.

- On observe ici un nuage de points représentant le prix des maisons par unité de surface en fonction du nombre de supérettes de proximité, à partir des données immobilières de Taïwan.

- Estimer l'ordonnée à l'origine de la droite de régression linéaire.

Estimation de l'ordonnée à l'origine (*intercept*)

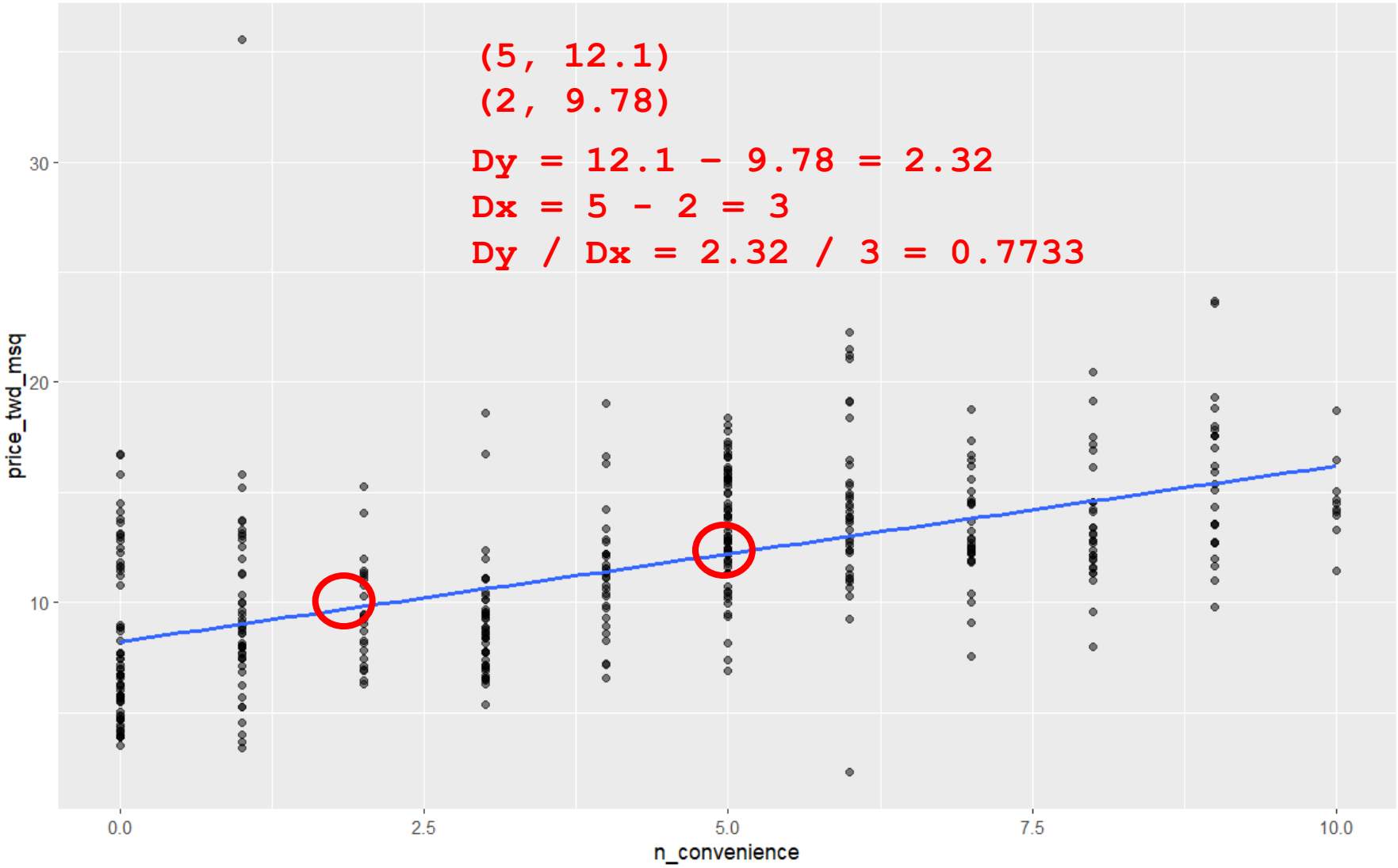


Estimation de la pente (*slope*)

- Voici le même nuage de points représentant le prix des maisons par unité de surface en fonction du nombre de commerces de proximité, à partir des données immobilières taïwanaises.

- Cette fois-ci, estimer la pente de la droite de tendance. Cliquez une fois sur la droite, puis double-cliquez à différents endroits pour relever la valeur de la pente.

Estimation de la pente (*slope*)



Régression linéaire avec `lm()`

- Bien que **`ggplot`** puisse afficher une droite de tendance de régression linéaire à l'aide de **`geom_smooth()`**, il ne permet pas d'accéder à l'ordonnée à l'origine et à la pente en tant que variables, ni de manipuler les résultats du modèle comme des variables. Il est donc parfois nécessaire d'effectuer soi-même une régression linéaire.

- On effectue une régression linéaire avec **`price_twd_msq`** comme variable de réponse, **`n_convenience`** comme variable explicative et **`taiwan_real_estate`** comme ensemble ou jeu de données.

- On utilise la fonction **`lm()`**.

Régression linéaire avec `lm()`

```
# Effectuer une régression linéaire de price_twd_msq
en fonction de n_convenience

lm(formula = price_twd_msq ~ n_convenience, data =
taiwan_real_estate)


Call:
lm(formula = price_twd_msq ~ n_convenience, data =
taiwan_real_estate)

Coefficients:
  (Intercept)  n_convenience
       8.2242         0.7981
```

Equation:

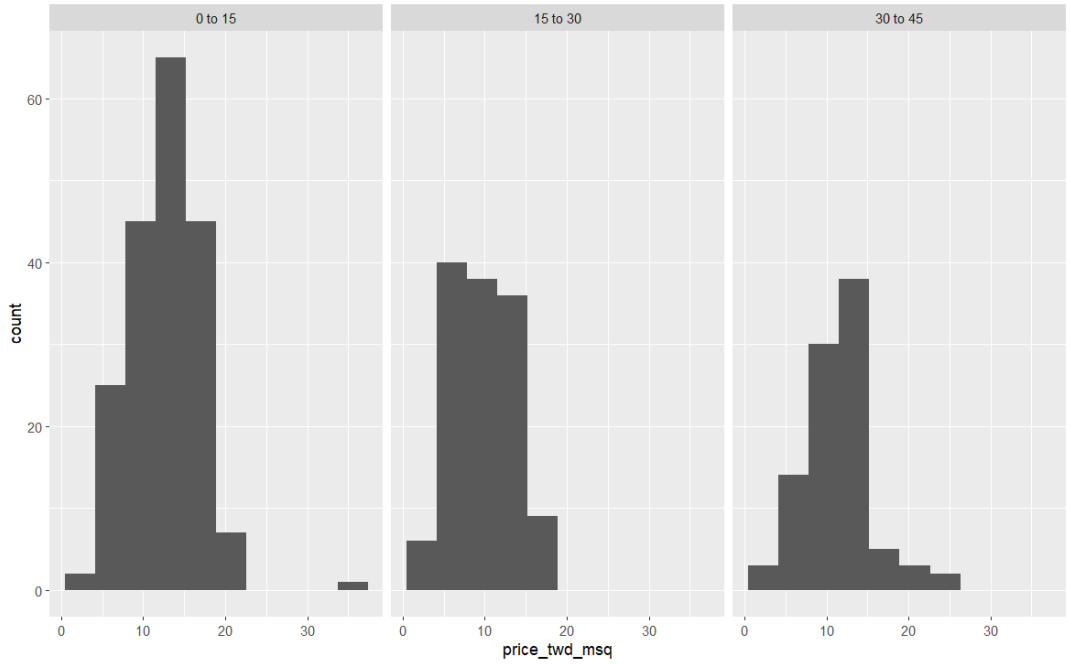
```
price_twd_msq = 8.2242 + 0.7981 * n_convenience
```

Visualisation de données numériques et catégoriques

- Si la variable explicative est catégorielle, le nuage de points utilisé précédemment pour visualiser les données n'est pas approprié. Il est préférable de tracer un histogramme pour chaque catégorie.
- L'ensemble de données sur l'immobilier taïwanais comporte une variable catégorielle : l'âge de chaque maison. Les âges sont répartis en trois groupes : 0 à 15 ans, 15 à 30 ans et 30 à 45 ans.

Visualisation de données numériques et catégoriques

```
# Représenter le price_twd_msq sous forme d'histogramme
ggplot(taiwan_real_estate, aes(x = price_twd_msq)) +
# Créer un histogramme à 10 classes.
geom_histogram(bins = 10) +
# Organiser le graphique de sorte que chaque groupe d'âge ait son propre panneau.
facet_wrap(vars(house_age_years))
```



Calcul des moyennes par catégorie

- Une bonne méthode pour explorer les variables catégoriques consiste à calculer des statistiques descriptives telles que la moyenne pour chaque catégorie. Ici, nous examinerons les moyennes groupées des prix des maisons.

```
taiwan_real_estate %>%
  # Grouper par house_age_years
  group_by(house_age_years) %>%
  # Résumer pour calculer le prix moyen des maisons
par unité de surface
  summarize(mean_price_by_group = mean(price_twd_msq))
house_age_years       mean_price_by_group
  <ord>                             <dbl>
1 0 to 15                            12.6
2 15 to 30                           9.88
3 30 to 45                           11.4
```

La fonction `lm()` avec une variable explicative catégorique

- Les régressions linéaires fonctionnent également avec des variables explicatives catégoriques. Dans ce cas, le code permettant d'exécuter le modèle est identique, mais les coefficients renvoyés diffèrent.

```
# Effectuer une régression linéaire de price_twd_msq
vs. house_age_years
lm(formula = price_twd_msq ~ house_age_years, data =
taiwan_real_estate)
Call:
lm(formula = price_twd_msq ~ house_age_years, data =
taiwan_real_estate)

Coefficients:
      (Intercept)  house_age_years.L  house_age_years.Q
          11.3025            -0.8798             1.7462
```

La fonction `lm()` avec une variable explicative catégorique

- On remarque que:
  - Il n'y a pas de coefficient pour la première catégorie.
  - Le coefficient de chaque catégorie est calculé par rapport à l'ordonnée à l'origine.
- Heureusement, on peut y remédier en modifiant légèrement la formule pour ajouter « + 0 ». On précise ainsi que tous les coefficients doivent être donnés par rapport à 0.
- Les coefficients ne sont que la moyenne pour chaque catégorie.

```
lm(formula = price_twd_msq ~ house_age_years + 0,
data = taiwan_real_estate)
Call:
lm(formula = price_twd_msq ~ house_age_years + 0, data = taiwan_real_estate)

Coefficients:
 house_age_years0 to 15  house_age_years15 to 30  house_age_years30 to 45
                 12.637                    9.877                   11.393
```

Faire des prédictions

- Le principal avantage de l'utilisation de modèles plutôt que du simple calcul de statistiques descriptives est que les modèles permettent d'effectuer des prédictions.

```
mdl_price_vs_conv <- lm(formula = price_twd_msq ~
n_convenience, data = taiwan_real_estate)

new_data <- data.frame(n_convenience = 0:10)

predict(mdl_price_vs_conv, new_data)
```

```
 1         2         3         4         5         6         7
8.224237  9.022317  9.820397 10.618477 11.416556 12.214636 13.012716

8         9         10        11
13.810795 14.608875 15.406955 16.205035
```

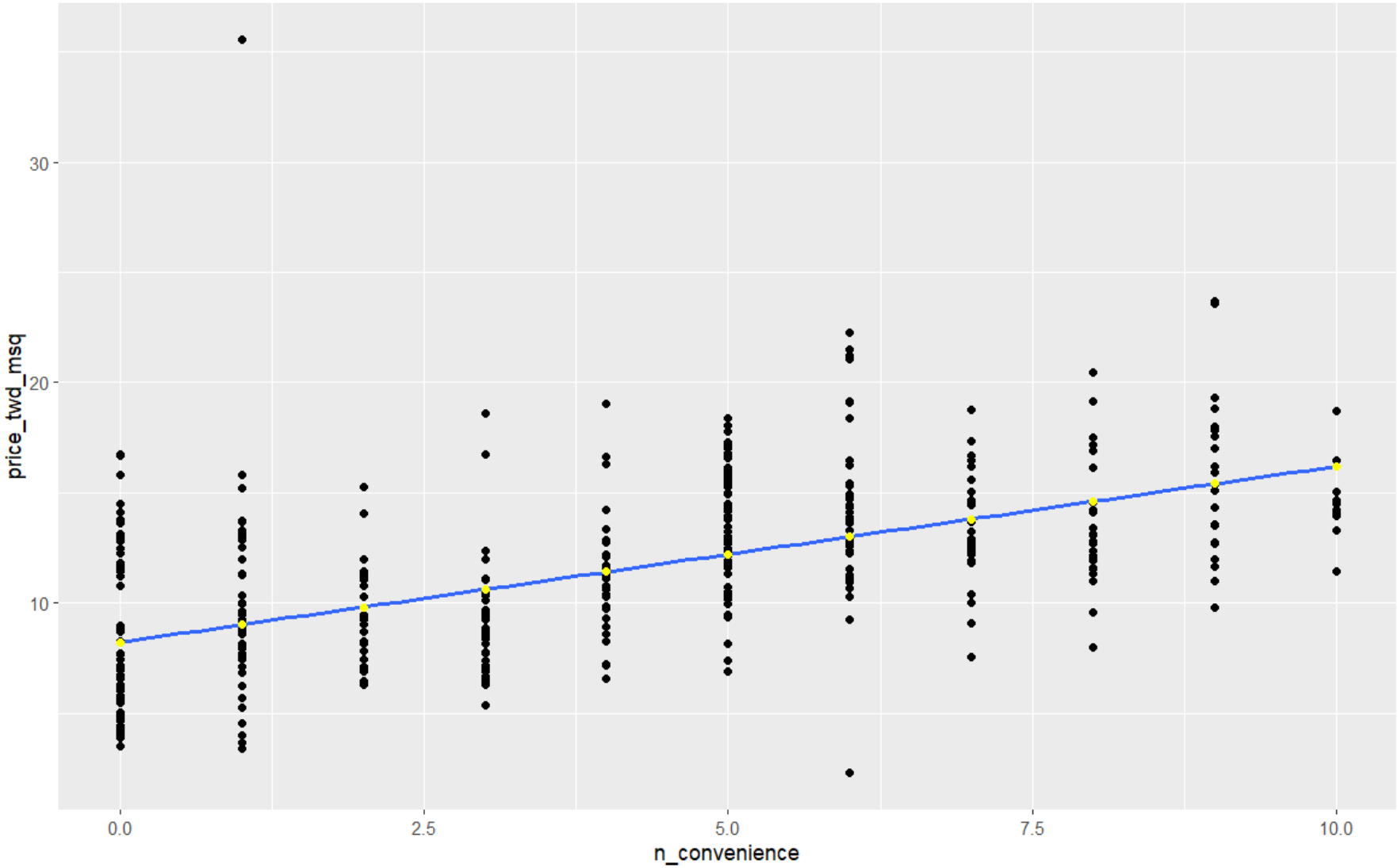
Visualisation des prédictions

```
prediction_data <- new_data

prediction_data <- prediction_data %>%
 mutate(
  price_twd_msq = predict(mdl_price_vs_conv, new_data)
 )
```

- Les données de prédiction que nous avons calculées comportent une colonne de valeurs de variables explicatives et une colonne de valeurs de variables de réponse. Nous pouvons donc les représenter sur le même nuage de points que les valeurs de la variable de réponse en fonction des variables explicatives.

Visualisation des prédictions



Les limites de la prédiction

- Dans le dernier exemple, nous avons effectué des prédictions sur des situations plausibles, susceptibles de se produire dans la réalité. Autrement dit, les cas où le nombre de supérettes à proximité était compris entre zéro et dix. Pour tester les limites de la capacité de prédiction du modèle, essayer des situations impossibles.

```
explanatory_data_2 <- data.frame(n_convenience = c(-1, 2.5))

predict(mdl_price_vs_conv, explanatory_data_2)

       1        2
 7.426158 10.219437
```

4.2. Régression Linéaire Multiple

Régression linéaire mutiple

- Un modèle de régression linéaire multiple est de la forme:

$$Y = \beta_0 + \beta_1 . X_1 + \beta_2 . X_2 + \ \ldots + + \beta_p . X_p + \epsilon$$

- Où:
  - Y est la variable dépendante (variable réponse)
  - $\beta_0, \beta_1 \ldots$ et β_p sont les coefficients
  - $X_1, X_2 \ \ldots et \ X_p$ sont les variables indépendantes (variables explicatives)
  - ϵ est une erreur aléatoire

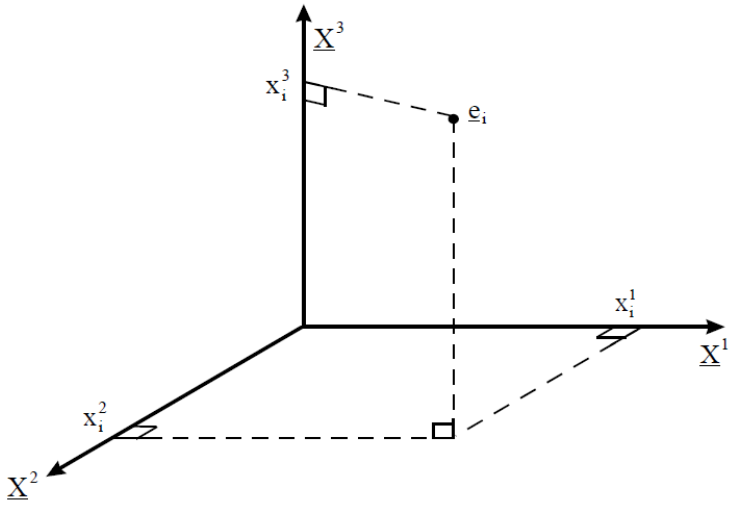
4.3. Analyse en composantes principales (ACP)

Analyse en composantes principales (ACP)

- L'Analyse en Composantes Principales (en anglais: Principal Component Analysis – PCA) est une technique de réduction de la dimensionnalité (Dimensionality Reduction) applicable dans:
  - L'analyse exploratoire de données (EDA)
  - Visualisation de données
  - Pré-traitement de données.

Analyse en composantes principales (ACP)

- L'ACP permet d'explorer les liaisons entre variables et les ressemblances entre individus.
- **On cherche à représenter le nuage des individus.**
  - A chaque individu noté $\boldsymbol{e_i}$, on peut associer un point dans $\boldsymbol{R^p}$ = espace des individus.
  - A chaque variable du tableau X est associé un axe de $\boldsymbol{R^p}$.

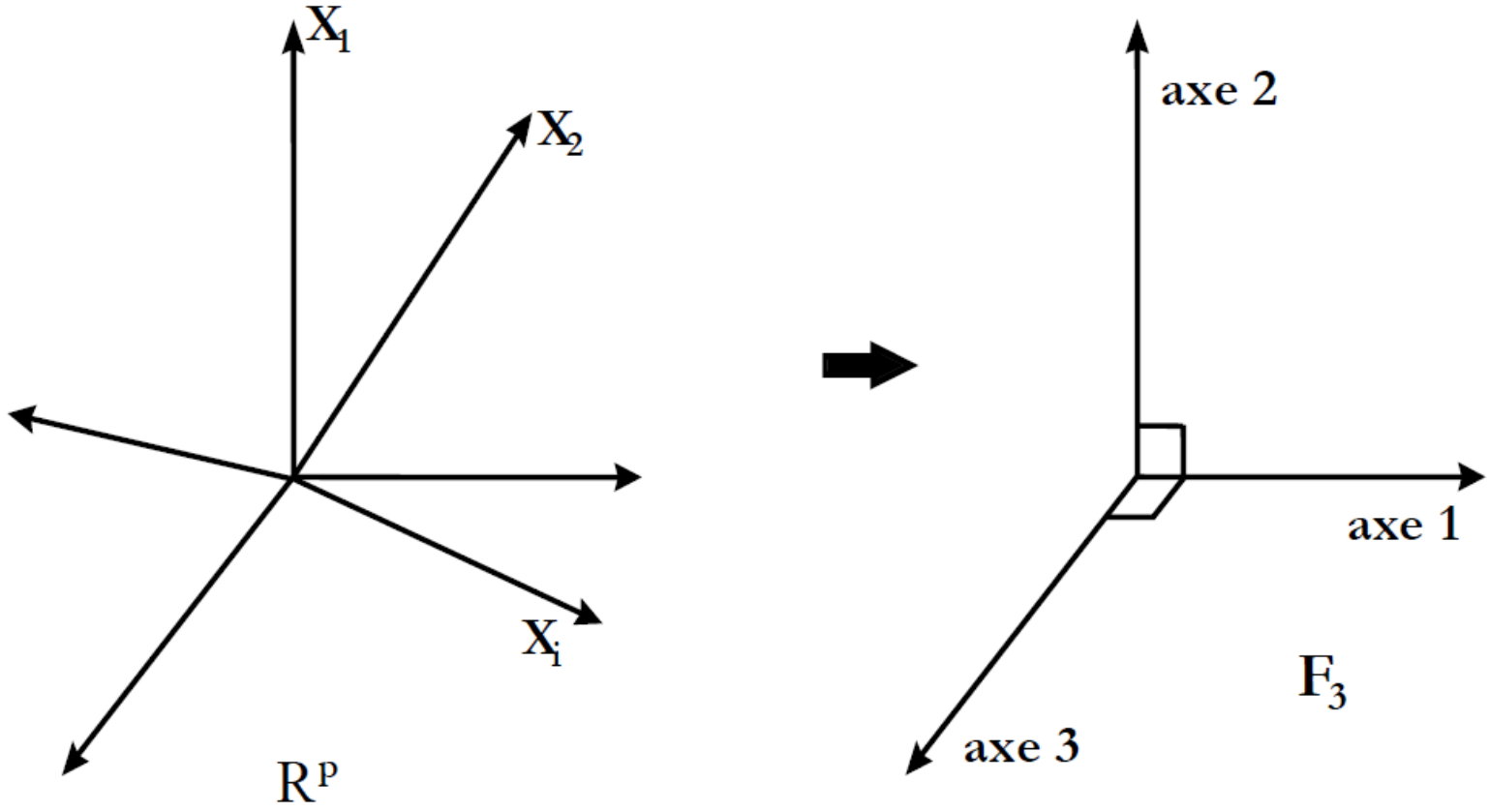


Principe de l'ACP

- On cherche une représentation des n individus , dans un sous-espace F_k de R^p de dimension k (k est petit 2, 3 …; par exemple un plan)
- Autrement dit, on cherche à définir k **nouvelles variables** obtenues comme des combinaisons linéaires des p variables initiales qui feront perdre le moins d'information possible.

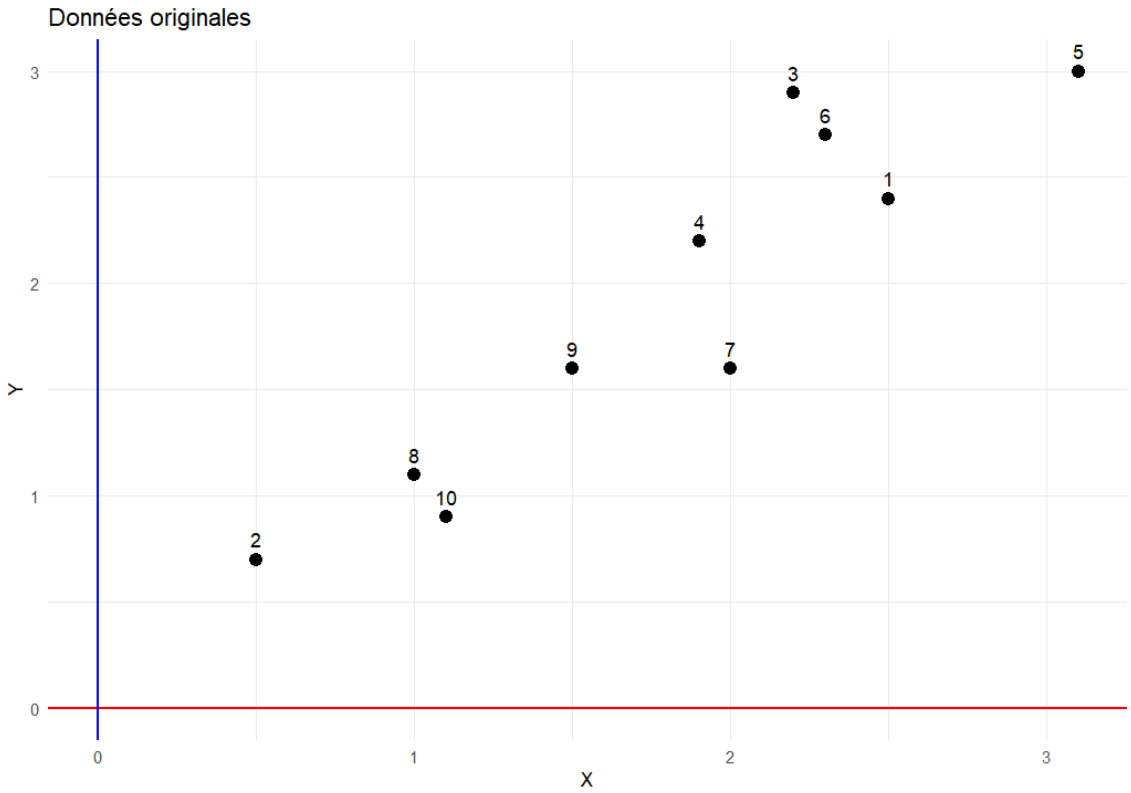
- Ces variables seront appelées ***composantes principales***
- les axes qu'elles déterminent : ***axes principaux***

Axes principaux



ACP: Exemple pratique (étape par étape)

- Considérons cet ensemble de données:
  - 10 observations
  - 2 variables



X	Y
2.5	2.4
0.5	0.7
2.2	2.9
1.9	2.2
3.1	3.0
2.3	2.7
2.0	1.6
1.0	1.1
1.5	1.6
1.1	0.9

Etape 1: Calculer la moyenne des variables

- On calcule la moyenne de chaque variable :

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

$$\bar{Y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

- En R

```
colMeans(df)

   X    Y
1.81 1.91
```

Etape 2: Centrer les données

- On soustrait la moyenne de chaque variable à chaque observation :

$$x'_i = x_i - \bar{X}$$
$$y'_i = y_i - \bar{Y}$$

- En R :

```
df_centered <- scale(df, center = TRUE, scale = FALSE)
df_centered
```

- Par exemple, pour le premier individu :

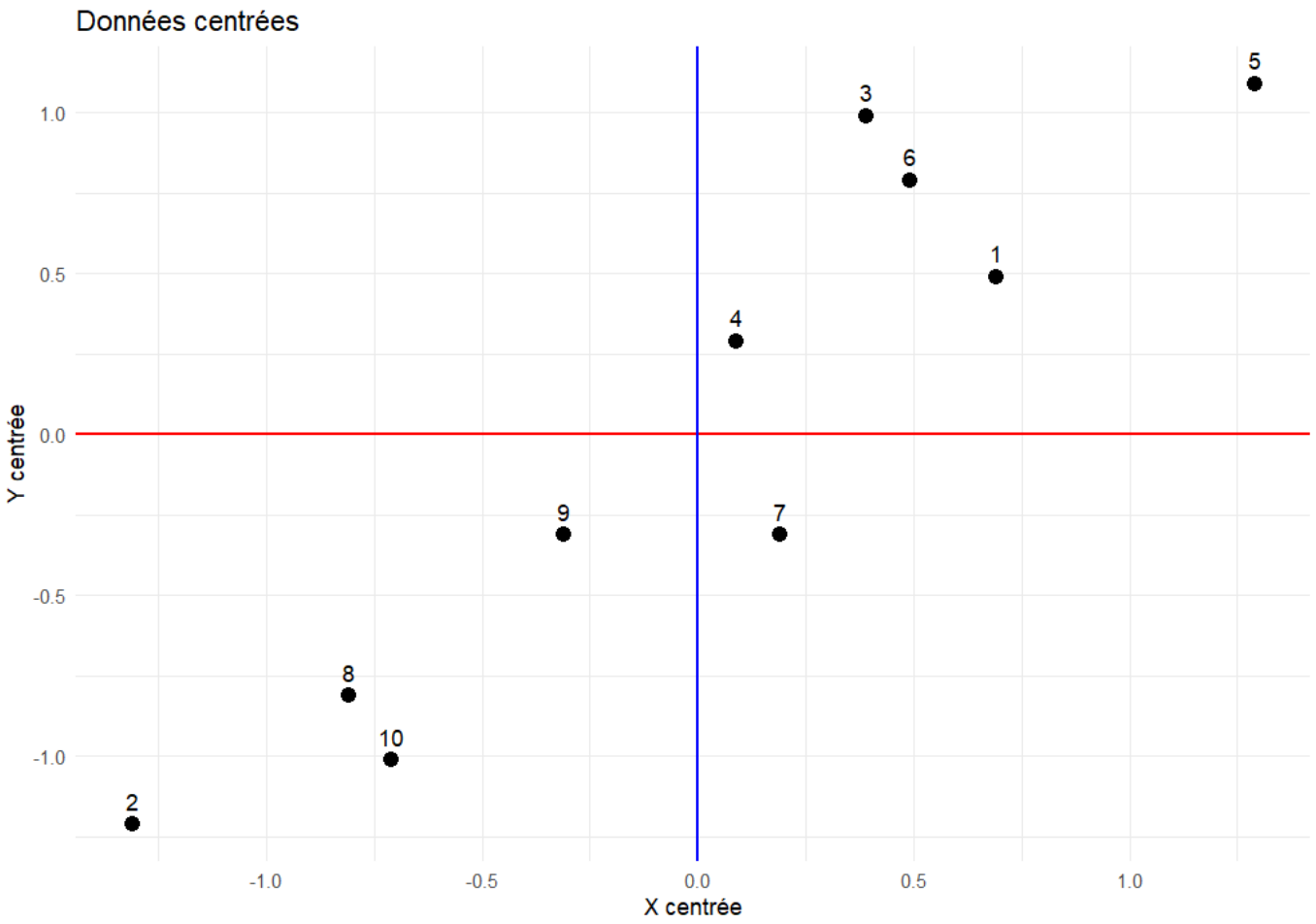
$$x'_1 = 2.5 - 1.81 = 0.69$$
$$y'_1 = 2.4 - 1.91 = 0.49$$

	X	Y
1	0.69	0.49
2	-1.31	-1.21
3	0.39	0.99
4	0.09	0.29
5	1.29	1.09
6	0.49	0.79
7	0.19	-0.31
8	-0.81	-0.81
9	-0.31	-0.31
10	-0.71	-1.01

Pourquoi centrer ?
Parce que l'ACP cherche les directions dans lesquelles les données varient le plus. On veut donc travailler autour de l'origine, c'est-à-dire autour de la moyenne.

Etape 2: Centrer les données

	X	Y
1	0.69	0.49
2	-1.31	-1.21
3	0.39	0.99
4	0.09	0.29
5	1.29	1.09
6	0.49	0.79
7	0.19	-0.31
8	-0.81	-0.81
9	-0.31	-0.31
10	-0.71	-1.01



Etape 3: Construire la matrice de covariance

- A partir des données centrées, on calcule la matrice de covariance $\boldsymbol{S}$:

$$S = \begin{bmatrix} cov(X,X) & cov(X,Y) \\ cov(Y,X) & cov(Y,Y) \end{bmatrix}$$

$$cov(X,X) = \frac{\sum(x_i - \bar{X}) * (x_i - \bar{X})}{n-1}$$

$$cov(X,Y) = \frac{\sum(x_i - \bar{X}) * (y_i - \bar{Y})}{n-1}$$

- En R:

```
S <- cov(df_centered)
S
```

Etape 3: Construire la matrice de covariance

$$S = \begin{bmatrix} 0.6165556 & 0.6154444 \\ 0.6154444 & 0.7165556 \end{bmatrix}$$

Interprétation

- Les variances sont :

$$Var(x) = 0.6166$$
$$Var(y) = 0.7166$$

- Et surtout :

$$Cov(x, y) = 0.6154$$

La covariance est fortement positive : **x et y ont tendance à augmenter ensemble**.

Etape 4: Calculer les valeurs propres

- Après avoir centré les données, nous avons calculé la matrice de covariance.
- Maintenant l'ACP cherche:
  - la direction dans laquelle le nuage varie le plus ➔ CP1
  - la direction perpendiculaire dans laquelle il reste le plus de variation ➔ CP2.
- C'est précisément ce que nous obtenons avec les **valeurs propres** (eigen values) et les **vecteurs propres** (eigen vectors).

Etape 4: Calculer les valeurs propres

- On cherche les **valeurs propres** de la matrice de covariance :

$$Sv = \lambda v$$

- C'est quoi ***v***?
  - On cherche un vecteur: $v = \begin{pmatrix} v_1 \\ v_2 \end{pmatrix}$
  - Ce vecteur représente **une direction dans notre espace initial** (X,Y).
- On connaît S, On cherche donc:
  - v: **la direction**
  - λ: **la variance dans cette direction**
- Le vecteur propre v donne la direction d'une composante principale, tandis que la valeur propre λ donne la variance expliquée par cette direction.

Etape 4: Calculer les valeurs propres

- **Comment trouver λ_1 et λ_2?**
  - Nous partons de: $Sv = \lambda v$
  - On peut écrire: $Sv - \lambda v = 0$
  - Comme: $v = Iv$ (I: matrice identité)
  - on obtient: $Sv - \lambda Iv = 0$
  - donc: $(S - \lambda I)\, v = 0$
- Pour qu'il existe un vecteur v **non nul**, il faut que: $\det(S - \lambda I) = 0$
- C'est cette équation qui nous permet de trouver les **valeurs propres** λ.

Etape 4: Calculer les valeurs propres

- Notre matrice est:

$$S = \begin{pmatrix} 0.6166 & 0.6154 \\ 0.6154 & 0.7166 \end{pmatrix}$$

- La matrice identité est:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

- Donc:

$$S - \lambda I = \begin{pmatrix} 0.6166 - \lambda & 0.6154 \\ 0.6154 & 0.7166 - \lambda \end{pmatrix}$$

- Nous devons avoir: $\det(S - \lambda I) = 0$

Calcul du déterminant

- Pour une matrice :

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

- Le determinant est:

$$ad - bc$$

- Donc:

$$(0.6166 - \lambda)(0.7166 - \lambda) - (0.6154)^2 = 0$$

→ $0.4418 - 0.6166\lambda - 0.7166\lambda + \lambda^2 - 0.3788 = 0$

→ $\lambda^2 - 1.3331\lambda + 0.0630 = 0$

- Nous avons maintenant une **équation du second degré**.

Résoudre l’ équation

- Nous avons :

$$\lambda^2 - 1.3331\lambda + 0.0630 = 0$$

- En utilisant :

$$\lambda = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

- on obtient les deux solutions :

$$\lambda_1 \approx 1.2840$$

- Et

$$\lambda_2 \approx 0.0491$$

- Ce sont les **deux valeurs propres**.

Pourquoi y a-t-il deux valeurs propres ?

- Parce que nous avons **deux variables** :

$$X, Y$$

- Notre espace est donc de dimension 2.
- L'ACP cherche donc deux axes : CP1 et CP2
- Chaque axe possède :
  - un **vecteur propre** (v) : direction
  - une **valeur propre** (λ) : variance dans cette direction.
- On peut résumer :

	Direction	**Variance**
CP1	v_1	$\lambda_1 = 1.2840$
CP2	v_2	$\lambda_2 = 0.0491$

Etape 4: Calculer les valeurs propres

- En R :

```
eigen_result <- eigen(S)
eigen_result$values
```

Etape5: Les vecteurs propres

- Nous avons trouvé :

$$\lambda_1 = 1.2840$$

- Nous retournons dans :

$$Sv = \lambda$$

- Donc :

$$(S - \lambda_1 I)v_1 = 0$$

- On remplace λ_1 :

$$\begin{pmatrix} 0.6166 - 1.2840 & 0.6154 \\ 0.6154 & 0.7166 - 1.2840 \end{pmatrix} \begin{pmatrix} v_{11} \\ v_{12} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

- soit approximativement :

$$\begin{pmatrix} -0.6674 & 0.6154 \\ 0.6154 & -0.5674 \end{pmatrix} \begin{pmatrix} v_{11} \\ v_{12} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Les vecteurs propres

- La première équation donne :

$$-0.6674\, v_{11} + 0.6154\, v_{12} = 0$$

- donc:

$$v_{12} \approx 1.0845\, v_{11}$$

- On peut choisir une valeur arbitraire pour v_{11}(par exemple 1) puis normaliser le vecteur.

$$v_1 = \begin{pmatrix} 1 \\ 1.0845 \end{pmatrix}$$

$$v_1 = \begin{pmatrix} \frac{1}{\sqrt{(1^2+1.0845^2}} \\ \frac{1.0845}{\sqrt{(1^2+1.0845^2}} \end{pmatrix} = \begin{pmatrix} 0.6779 \\ 0.7352 \end{pmatrix}$$

- C'est la direction de **CP1**.

Et v_2

- On fait exactement la même chose avec :

$$\lambda_2 = 0.0491$$

- Nous retournons dans :

$$Sv = \lambda$$

- Donc :

$$(S - \lambda_2 I)v_2 = 0$$

- On remplace λ_1 :

$$\begin{pmatrix} 0.6166 - 0.0491 & 0.6154 \\ 0.6154 & 0.7166 - 0.0491 \end{pmatrix} \begin{pmatrix} v_{21} \\ v_{22} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

- soit approximativement :

$$\begin{pmatrix} 0.5675 & 0.6154 \\ 0.6154 & 0.6675 \end{pmatrix} \begin{pmatrix} v_{21} \\ v_{22} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

Les vecteurs propres

- La première équation donne :

$$0.5675v_{21} + 0.6154v_{22} = 0$$

- donc:

$$v_{21} \approx -1.0844\, v_{22}$$

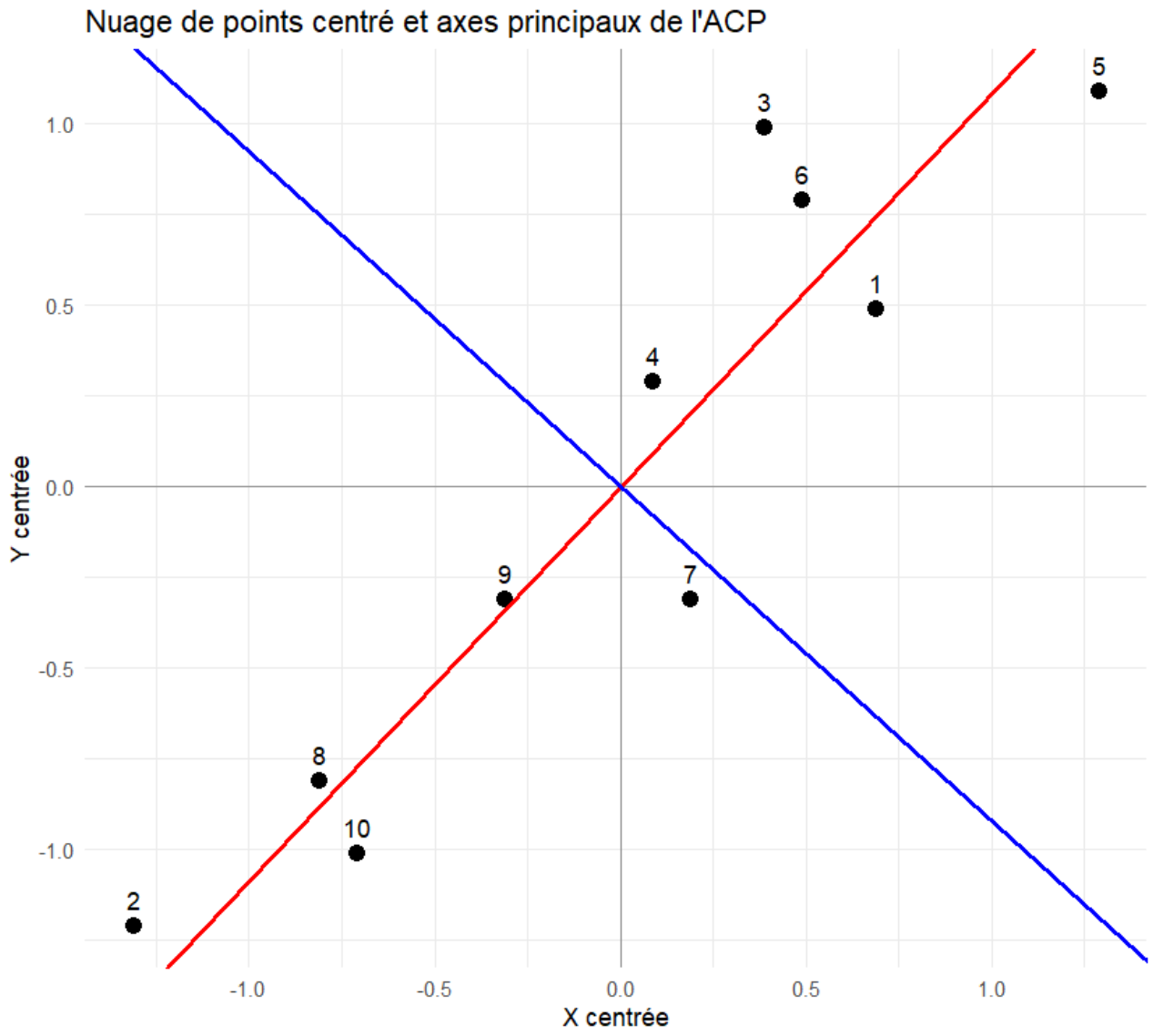
- On peut choisir une valeur arbitraire pour v_{21}(par exemple 1) puis normaliser le vecteur.

$$v_2 = \begin{pmatrix} -1.0844 \\ 1 \end{pmatrix}$$

- $$v_2 = \begin{pmatrix} \frac{-1.0844}{\sqrt{(-1.0844)^2+1^2}} \\ \frac{1}{\sqrt{(-1.0844)^2+1^2}} \end{pmatrix} = \begin{pmatrix} -0.7352 \\ 0.6779 \end{pmatrix}$$

- C'est la direction de **CP2**.

Les axes principaux



Les axes principaux

- Axe rouge (CP1) : premier axe principal, associé à la plus grande valeur propre $\lambda_1 \approx 1.2840$. Il explique environ 96,32 % de la variance totale.
- Axe bleu (CP2) : deuxième axe principal, associé à $\lambda_2 \approx 0.0491$. Il explique environ 3,68 % de la variance totale.

- 1.284/(1.284+0.0491) = 0.96316 = 69.32 %
- 0.0491/(1.284+0.0491) = 0.03683 = 3.68 %

Projeter les données sur les axes principaux

- Chaque axe principal est défini par un **vecteur propre** de la matrice de covariance.
- Les valeurs propres permettent de classer les axes selon la quantité de variance qu'ils expliquent.
- **Choix des axes :**
- **Premier axe (CP1)** : vecteur propre associé à la plus grande valeur propre λ_1. Il conserve le maximum de variance possible.
- **Deuxième axe (CP2)** : vecteur propre associé à la deuxième plus grande valeur propre λ_2. Il est orthogonal au premier axe.
- En général, on peut conserver les premiers axes qui expliquent une proportion suffisante de la variance totale.

Construire la transformation des données

- On utilise les vecteurs propres normalisés comme nouvelles directions de projection.
- Dans notre exemple, les vecteurs propres sont :

$$\begin{pmatrix} 0.6779 \\ 0.7352 \end{pmatrix} \text{ et } \begin{pmatrix} -0.7352 \\ 0.6779 \end{pmatrix}$$

- On les place en colonnes dans une matrice de transformation (P) :

$$\mathrm{p} = \begin{pmatrix} 0.6779 & -0.7352 \\ 0.7352 & 0.6779 \end{pmatrix}$$

- Soit (X_c) la matrice des données centrées, contenant les variables X et Y.

Construire la transformation des données

- La transformation s'effectue par multiplication matricielle :
- $Z = X_c P$
  - (X_c) : données centrées initiales.
  - (P) : matrice des vecteurs propres.
  - (Z) : données exprimées dans le nouveau repère des composantes principales.

	X	Y
1	0.69	0.49
2	-1.31	-1.21
3	0.39	0.99
4	0.09	0.29
5	1.29	1.09
6	0.49	0.79
7	0.19	-0.31
8	-0.81	-0.81
9	-0.31	-0.31
10	-0.71	-1.01

$$\begin{pmatrix} 0.6779 & -0.7352 \\ 0.7352 & 0.6779 \end{pmatrix}$$

0.69 * 0.6779 + 0.49 * 0.7352 = 0.827999

0.69 * (-0.7352) + 0.49 * 0.6779 = -0.175117

Construire la transformation des données

- Pour le premier individu, les valeurs initiales sont (X=2.5) et (Y=2.4). Après centrage, on obtient (X_c =0.69) et (Y_c =0.49).
- Sa projection sur le premier axe vaut :
- CP1=0.6779(0.69)+0.7352(0.49) = 0.829
- Sa projection sur le deuxième axe vaut :
- CP2=-0.7352(0.69)+0.6779(0.49) = -0.175
- L'individu est donc représenté par les coordonnées approximatives (0.829,-0.175) dans le nouveau repère. Si l'on ne conserve que CP1, il est représenté par la seule coordonnée (0.829).

L'ACP pas à pas sous R

```
df <- data.frame(
X = c(2.5, 0.5, 2.2, 1.9, 3.1, 2.3, 2.0, 1.0, 1.5, 1.1),
Y = c(2.4, 0.7, 2.9, 2.2, 3.0, 2.7, 1.6, 1.1, 1.6, 0.9))

colMeans(df)

df_centered <- scale(df, center = TRUE, scale = FALSE)

eigen_result <- eigen(S)
eigen_result$values
eigen_result$vectors

lamda1 <- eigen_result$values[1]
lamda2 <- eigen_result$values[2]
v1 <- eigen_result$vectors[,1]
v2 <- eigen_result$vectors[,2]
```

L'ACP pas à pas sous R

```
library(ggplot2)
ggplot(df_centered, aes(x = X, y = Y)) +
  geom_hline(yintercept = 0, color = "grey60") +
  geom_vline(xintercept = 0, color = "grey60") +
  geom_point(size = 3) +
  geom_text(aes(label = 1:nrow(df_centered)),vjust = -0.8) +

  # Premier axe principal
  geom_abline(
    slope = v1[2] / v1[1],
    intercept = 0,
    color = "red",
    linewidth = 1
  ) +
```

L'ACP pas à pas sous R

```
  # Deuxième axe principal
  geom_abline(
    slope = v2[2] / v2[1],
    intercept = 0,
    color = "blue",
    linewidth = 1
  ) +

  labs(
    title = "Nuage de points centré et axes principaux de
l'ACP",
    x = "X centrée",
    y = "Y centrée"
  ) +

  theme_minimal()
```

ACP sous R avec `prcomp`

```
library(ggplot2)

# ACP
res <- prcomp(df, scale. = TRUE)

# Ajouter les coordonnées des individus à df
df$PC1 <- res$x[, 1]
df$PC2 <- res$x[, 2]

# Ajouter les numéros des individus
df$Individu <- 1:nrow(df)
```

ACP sous R

```
# Graphique ACP
ggplot(df, aes(PC1, PC2)) +
  geom_point(size = 3, alpha = 0.8) +
  geom_text(aes(label = Individu), vjust = -0.8) +
  geom_hline(yintercept = 0, linewidth = 1, color =
"red") +
  geom_vline(xintercept = 0, linewidth = 1, color =
"blue") +
  labs(
    title = "ACP",
    x = "PC1",
    y = "PC2"
  ) +
  theme_minimal()
```